COMP 251 - Assignment 10: Sorting Algorithms

Question 1: Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Notes
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Good for small or nearly sorted arrays
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Few swaps, simple implementation
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	Usually fastest in practice
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Stable, requires extra memory
Bucket Sort	O(n+k)	O(n+k)	$O(n^2)$	Efficient when data is uniformly distributed

Trade-offs: Quick sort is usually fastest in practice. Merge sort is stable but needs extra memory. Insertion sort works well for almost sorted arrays. Selection sort is simple but inefficient. Bucket sort can be linear under suitable conditions.

Question 2:

Bucket Sort

Bucket sort is not in-place because it requires additional buckets (extra memory) to store elements before combining them back into the original array.

Question 3:

O(n) vs O(n log n) Sorting

If a list is already sorted, insertion sort runs in O(n) time because it only needs one comparison per element. Merge sort still performs O(n log n) operations regardless of the initial order. If the list is reversed, insertion sort degrades to $O(n^2)$, while merge sort remains O(n log n).

Question 4:

Almost Sorted Array

For an almost sorted array, insertion sort is usually preferred because it can run close to O(n) time when only a few elements are out of place.